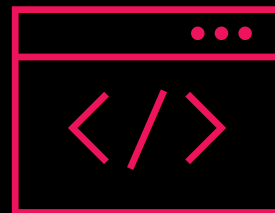


GOODWE, MATRIZES E EXERCÍCIOS

Pensamento Computacional e Automação com Python



Challenge GoodWe



- Mentorias!

- Já rolaram... (13/05 rem., 23 e 25/06 pres.)

- Bancas

1. **28/ago:** vídeo pitch/técnico de 3 min
2. **Sem. 28/set:** banca remota para classificação ao NEXT
3. **Sem. 19/out:** banca presencial para classificação do TOP 3

* todas as datas estão sujeitas a modificação

Mais sobre os dias 23 e 25/06



1º SEMESTRE

1. Pensamento Computacional
2. Conceitos Básicos de Programação
3. Introdução ao Python e Instalação
4. Variáveis, Tipos Primitivos e Saída de dados
5. Operadores Aritméticos
6. Usando Módulos
7. Operadores relacionais
8. Operadores lógicos
9. Estruturas condicionais
10. Estruturas de repetição
11. Strings e Listas

2º SEMESTRE

1. Matrizes
2. Dicionários
3. Tuplas
4. Funções e recursividade
5. Modularização
6. Tratamento de erros
7. Arquivos
8. Classes e Objetos
9. Herança e Polimorfismo



Checkpoints da disciplina (CP)

FIAP

Calendário 2º semestre:

CP1 – Até 28/08: 1ª Entrega de Projeto e Apresentação GoodWe (vídeo pitch 3 min)

CP2 – Semana de 31/08 e 01/09

CP3 – Semana de 13/10 (19/10 para 1CCPK-I-Q-W)

Obs.: sujeito a alterações



Sprints

FIAP

Calendário 2º semestre:

Sprint 3 – De 05/09 até 30/09

Sprint 4 – De 01/10 até 31/10

Obs.: sujeito a alterações



NEXT

FIAP

Calendário 2º semestre:

NEXT (24/10 – SÁBADO)

Normalmente a partir das 14h para visitantes e a partir das ~10h/11h para expositores.

Infos oficiais irão sair por aqui: <https://www.fiap.com.br/next/>

Obs.: sujeito a alterações



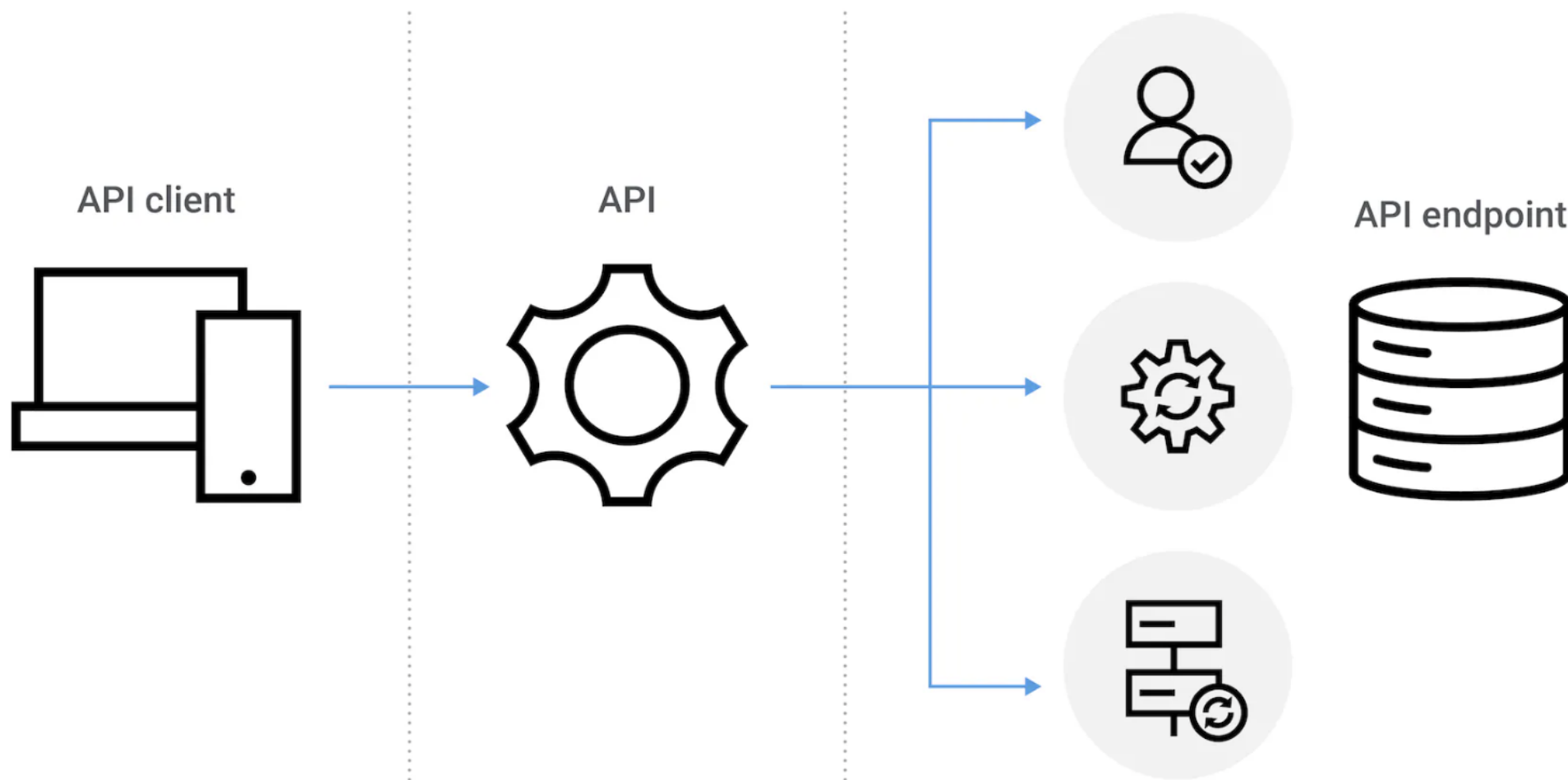
AVALIAÇÃO GLOBAL SOLUTION (GS) 2º SEMESTRE

FIAP

INÍCIO: **04 de NOV. a 19 de NOV.**

Obs.: sujeito a alterações

API – Pesquisem em detalhes (extra)



REVISÃO: DESAFIO API em Alerta

- Uma API possui três endpoints. A matriz abaixo registra os códigos HTTP das últimas requisições realizadas:

```
endpoints = ["/login", "/produtos", "/pedidos"]
```

```
status = [  
    [200, 200, 401, 200, 500],  
    [200, 200, 200, 200, 200],  
    [201, 500, 502, 201, 500]  
]
```

- Cada **linha** representa um endpoint e cada **coluna**, uma requisição.

REVISÃO: DESAFIO API em Alerta

Crie um programa que:

1. calcule a porcentagem de requisições bem-sucedidas de cada endpoint;
2. identifique o endpoint com mais erros;
3. verifique se algum endpoint teve dois erros seguidos;
4. classifique cada endpoint como:
 - ESTÁVEL: pelo menos 80% de sucesso;
 - INSTÁVEL: menos de 80%;
 - CRÍTICO: dois erros consecutivos.

Considere sucesso qualquer código entre 200 e 299.
O programa deve utilizar pelo menos uma função e funcionar caso novos endpoints ou requisições sejam adicionados.



EXERCÍCIOS

Revisão 1º Semestre

EXERCÍCIO 1 - WHILE

- Faça um programa que exiba a mensagem "Olá, Mundo".
- Essa mensagem deverá ser exibida repetidamente.
- Ao final de toda iteração da repetição, você deve perguntar ao usuário se ele deseja exibir a mensagem novamente.
- Se sim, exiba novamente. Senão, saia do loop e exiba a mensagem "Fim".

EXERCÍCIO 2 – FOR

- Contagem de 0 a 100 pulando de 10 em 10.
- O terminal deve ficar assim:

```
0
10
20
30
40
50
60
70
80
90
100
```

EXERCÍCIO 3

- Escreva um programa que dado um inteiro n **positivo** calcula e imprime a soma de todos os números inteiros entre 1 e n.
- Valide a entrada do usuário, só aceite números positivos!!
- Dica: use **while** para a validação e **for** para a soma.
- Por exemplo, se $n = 10$ então deverá ser calculado:
 - $1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 = 55$
- E a impressão final seria:
 - A soma de 1 até 10 é: 55

EXERCÍCIO 4

- Escreva um algoritmo que recebe um inteiro positivo n e imprime todos os divisores positivos de n .
- Utilize o laço `for`.
- Exemplo:
Suponha que $n = 28$, nessa situação devemos imprimir os números 1, 2, 4, 7, 14 e 28, que são todos os divisores do 28.
- Dica: para o número ser divisor de n , a divisão precisa ter **resto nulo**.

EXERCÍCIO 5

- Determine e mostre todos os números primos no intervalo de 2 a 2000.

Dicas:

- Para resolver esse problema, primeiro faça um algoritmo que verifica se um número inteiro qualquer é primo ou não.
- A seguir, com esse código em mãos, faça os ajustes necessários para mostrar todos os números primos no intervalo solicitado.
- Você precisará colocar uma estrutura de repetição dentro da outra.
 - Laços aninhados!!!!

EXERCÍCIO 6

- Escreva um algoritmo que recebe um número inteiro $n > 0$, cria um vetor de números reais com n posições e preenche o vetor com n números aleatórios reais.
- Depois de preenchido o vetor, imprima na tela todos os números gerados.

EXERCÍCIO 7

- Escreva um algoritmo que lê um número inteiro $n > 0$ e preenche um vetor de caracteres de n posições.
- Depois de preencher o vetor, você deverá inverter o seu conteúdo, ou seja, trocar o conteúdo da primeira posição (0) com a última ($n - 1$) a segunda com a penúltima e assim por diante até que o vetor esteja invertido.

EXERCÍCIO 8

- Faça um programa que realize a soma de duas matrizes, com **mesmas dimensões**. Seu programa deve ter 2 matrizes A e B de números inteiros. A terceira matriz deve ser a soma de A com B.
- Exemplo de soma:

$$\begin{bmatrix} -3 & 5 & 2 \\ 1 & 6 & 4 \end{bmatrix} + \begin{bmatrix} 7 & 2 & 0 \\ 9 & -2 & 3 \end{bmatrix} = \begin{bmatrix} 4 & 7 & 2 \\ 10 & 4 & 7 \end{bmatrix}$$

ALGUMA DÚVIDA?

Entre em contato pelo Teams ou por e-mail



Alexandre Russi Junior

Coordinator of Technology and Educational Projects |
Higher Education Professor | Artificial Intelligence



[/alexandrerrussi](#)